

Machine Learning for Design



Assignment 3

Group: G 4

Members: Vlad Dobriakov, Josha Leemans, Karlijn Tangelder, Jop van der Post & Bing Zwikker

Introduction

In this project we developed a machine learning model that is able to classify the presence of heart disease using a combined dataset from the UCI repository (Switzerland and Hungary). During the data preprocessing we handled with missing values and removed values that were biologically impossible. This data cleaning process naturally transformed our target variable into binary values. We then trained and evaluated three models: a dummy classifier, K-Nearest Neighbours and a random forest classifier. To evaluate we used a 70/30 train-test split and a 3fold cross validation method. Our conclusion is that random forest demonstrated the best performance in recognizing complex patterns. Our analysis confirmed that features with high variance and medical relevance, such as maximum heart rate and ST depression, were the strongest predictors. Finally, to address class imbalance and further enhance model reliability, we proposed implementing SMOTE oversampling.

Data Preprocessing

Combining datasets

In the beginning we combined the two data sets together. Since both datasets originate from the same UCI Heart Disease dataset and share the same feature structure, they can be merged directly. Our primary motivation was to increase the number of samples, creating a more robust dataset for our machine learning model. However, since they come from different sources, we decided to add a dataset label before merging, to track the origin for each row and account for potential differences in feature distributions from the different hospitals. And at lastly we combined the 2 data sets in one data set named data without original index to prevent doubled index problems.

```
# Reading the data swiss =  
pd.read_csv('data/processed.switzerland.csv') hung =  
pd.read_csv('data/processed.hungarian.csv')  
  
# Add dataset label swiss["dataset"] =  
"swiss"  
hung["dataset"] = "hung"  
  
# Combine datasets  
data = pd.concat([swiss, hung], ignore_index=True)
```

After printing we saw that: each row represents a patient, and each column represents a medical feature such as age, blood pressure, cholesterol level, and heart-related measurements.

Based on the dataset description from the UCI repository, the features can be divided into numerical and categorical variables.

- **Numerical features include:** age, trestbps, chol, thalach, oldpeak, and ca
- **Categorical features include:** sex, cp, fbs, restecg, exang, slope, and thal

Dealing with missing values

A key observation is that several features contain a large number of missing values, especially ca and thal columns were almost entirely empty. Since imputing so much missing data would introduce heavy bias, we decided to drop these columns entirely. This was done by using the code part:

```
data = data.drop(columns=["ca", "thal"])
```

After that we dropped the other NaN values by using:

```
data = data.dropna()
```

After that we had 141 rows of data that are clean and ready to be used.

Structuring data

Also there could be considered to change some in the feature description. The cp and slope features are categorical but encoded as integers. These should ideally be transformed using one-hot encoding to avoid misleading the model with artificial ordering.

Now that we have a clean dataset, we used the following code to get the data statistics:

```
data.describe()
```

This function is to compute the distribution of the numerical data. This function automatically calculates key statistical measures such as the mean, standard deviation, and median. Providing a clear overview of the central tendencies and spread within our features.

After this all we can conduct the following:

	age	sex	cp	trestbps	chol	fb	restecg	thalach	exang	oldpeak	slope	target
count	141.000000	141.000000	141.000000	141.000000	141.000000	141.000000	141.000000	141.000000	141.000000	141.000000	141.000000	141.000000
mean	52.602837	0.808511	3.553191	136.978723	176.510638	0.092199	0.283688	123.056738	0.702128	1.259574	1.900709	1.191489
std	7.931744	0.394876	0.814384	19.078585	135.604710	0.290337	0.497512	22.853188	0.458953	0.951614	0.364593	0.955547
min	31.000000	0.000000	1.000000	92.000000	0.000000	0.000000	0.000000	60.000000	0.000000	-1.000000	1.000000	0.000000
25%	48.000000	1.000000	3.000000	120.000000	0.000000	0.000000	0.000000	105.000000	0.000000	0.800000	2.000000	1.000000
50%	53.000000	1.000000	4.000000	135.000000	219.000000	0.000000	0.000000	124.000000	1.000000	1.000000	2.000000	1.000000
75%	57.000000	1.000000	4.000000	150.000000	273.000000	0.000000	1.000000	140.000000	1.000000	2.000000	2.000000	1.000000
max	73.000000	1.000000	4.000000	200.000000	603.000000	1.000000	2.000000	184.000000	1.000000	5.000000	3.000000	4.000000

The dataset now contains 141 samples. The average age of patients is approximately 52.6 years, with a range from 31 to 73, indicating a middle-aged population.

Resting blood pressure (trestbps) has a mean of 136.98 mm Hg, with values ranging from 92 to 200, showing moderate variation across patients.

The maximum heart rate (thalach) has a mean of 123.06, with values between 60 and 184, indicating variability in patient fitness or heart condition. The feature oldpeak, which represents ST depression, has a mean of 1.26 and ranges from -1 to 5, where negative values may indicate measurement inconsistencies.

Removing invalid values

Several categorical features, such as sex, fbs, and exang, are binary (0 or 1), while others such as cp, restecg, and slope contain a limited set of discrete values, confirming that they should be treated as categorical variables rather than continuous ones.

Further inspection of the features revealed data entry errors that required preprocessing. For instance, the chol (cholesterol) feature contains values equal to 0, which is biologically impossible,

and oldpeak contains negative values (-1). To solve this, we will remove the rows containing these invalid entries with the following code:

```
data = data[(data['chol'] > 0) & (data['oldpeak'] >= 0)]
```

This code results in the following table:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	target
count	95.000000	95.000000	95.000000	95.000000	95.000000	95.000000	95.000000	95.000000	95.000000	95.000000	95.000000	95.000000
mean	50.631579	0.747368	3.463158	137.515789	261.978947	0.094737	0.231579	127.631579	0.736842	1.640000	1.905263	0.715789
std	6.860390	0.436827	0.884987	18.682603	68.979951	0.294405	0.424079	20.989893	0.442683	0.796455	0.328559	0.453430
min	31.000000	0.000000	1.000000	92.000000	117.000000	0.000000	0.000000	82.000000	0.000000	0.000000	1.000000	0.000000
25%	47.000000	0.500000	3.000000	124.500000	217.500000	0.000000	0.000000	113.500000	0.000000	1.000000	2.000000	0.000000
50%	52.000000	1.000000	4.000000	138.000000	260.000000	0.000000	0.000000	128.000000	1.000000	1.500000	2.000000	1.000000
75%	55.000000	1.000000	4.000000	150.000000	293.000000	0.000000	0.000000	142.000000	1.000000	2.000000	2.000000	1.000000
max	65.000000	1.000000	4.000000	200.000000	603.000000	1.000000	1.000000	184.000000	1.000000	5.000000	3.000000	1.000000

As visible in the new table the wrong data is successfully deleted. The minimum of chol is now 117 and oldpeak is 0, which means that the impossible values are deleted. The amount of rows has decreased from 141 to 95, which means we deleted $141 - 95 = 46$ rows.

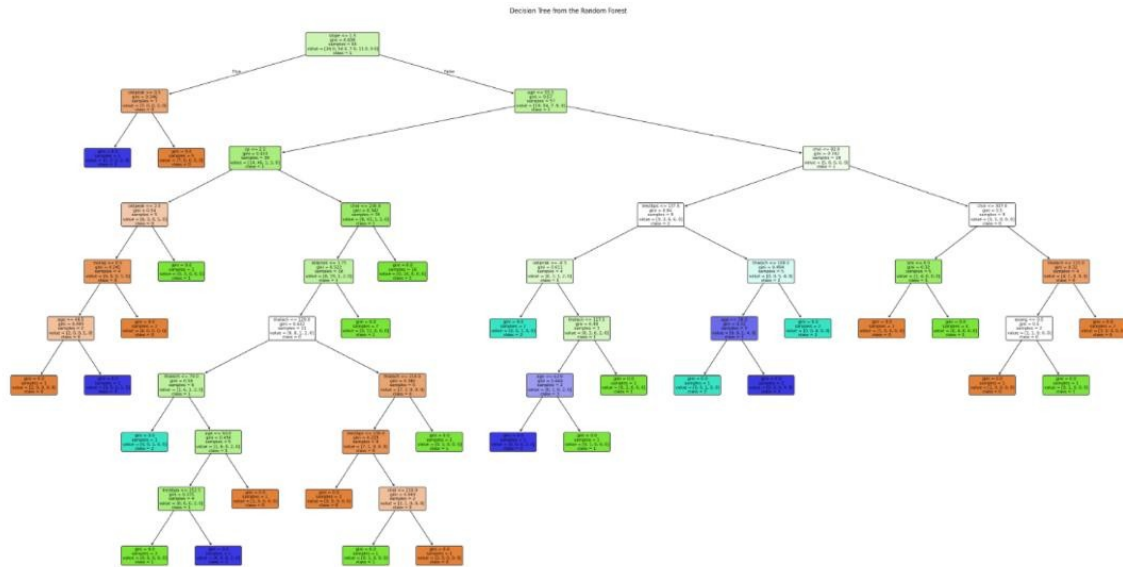
The target variable only has a minimum of 0 and a maximum of 1, which means we deleted the rows with target value 2, 3 or 4. This tells us that the people with a high level of heart disease were the people with the incorrect values of chol or oldpeak. We didn't make the target binary ourselves, but it's a side-effect of cleaning our data (with 0 = no heart disease and 1 = a present heart disease).

Model Training

Three different machine learning models were used to classify the presence of heart disease, namely: a Dummy Classifier, a K-Nearest Neighbours (KNN) classifier and a Random Forest classifier. These were chosen to represent an increasing level of complexity, ranging from a simple baseline to a model capable of identifying more complex patterns in the data. The Dummy Classifier was used as a reference point, as this model does not learn patterns from the dataset, but instead makes predictions based on a fixed rule. In this case, the “constant” strategy was chosen, whereby class 0 is predicted every time. The ultimate aim is to establish a baseline performance, so that it can be determined whether the other models are utilising any information from the data at all. The K-Nearest Neighbours classifier classifies the data points based on similarities. The distance to all points in the training data is calculated for all new data points, so that the k neighbours that are closest can then be selected. Here $k=4$ was chosen. The classification is ultimately determined by the class that occurs most frequently among these neighbours. A smaller value of k makes the model more susceptible to noise (overfitting), but a larger value can result in oversimplification (underfitting). Consequently, the choice of $k=4$ strikes a balance between the two extremes.

The Random Forest classifier is a more advanced model built on a collection of multiple decision trees. Each decision tree is trained on its own subset of the dataset and therefore makes an independent prediction. The final classification is determined by the class with the most predictions across all decision trees. In this study, we opted for a small number of trees, namely 4, so that the model can be kept computationally efficient and manageable, whilst still delivering better results than the simpler models. To better understand how decisions are made in a Random Forest, a visualisation was created for one of the decision trees. In such a tree, the data is split step by step based on features: for example, an initial split might occur on the “oldpeak” feature, where a check is made to see if the value exceeds a specific threshold. In this case, the data point is then sent to a branch where the probability of heart disease is higher. A subsequent split may then occur, such as on “thalach” (maximum heart rate). Through this series of decisions, a classification is ultimately made.

The visualisation of the decision trees shows that features such as “thalach” and “oldpeak” are used earlier in the decision-making process. This indicates that these variables hold considerable discriminatory power within the dataset. For example, higher values of “oldpeak” more frequently result in branches where the risk of heart disease is higher. This demonstrates that the model utilises relevant features to make predictions.

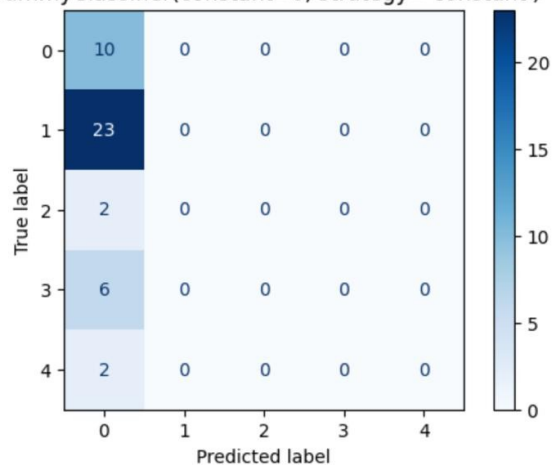


The model's learning processes were directly influenced by the pre-processing steps. Removing features with missing values, such as "thal" and "ca", reduced the complexity of the feature space. This reduces the likelihood of unreliable data causing the model to learn incorrect patterns, but may result in the loss of potentially relevant information. Unrealistic values, such as negative oldpeak and chol=0 values, have also been removed, making the dataset more consistent and, consequently, more reliable. This enables the models to better recognise relevant patterns

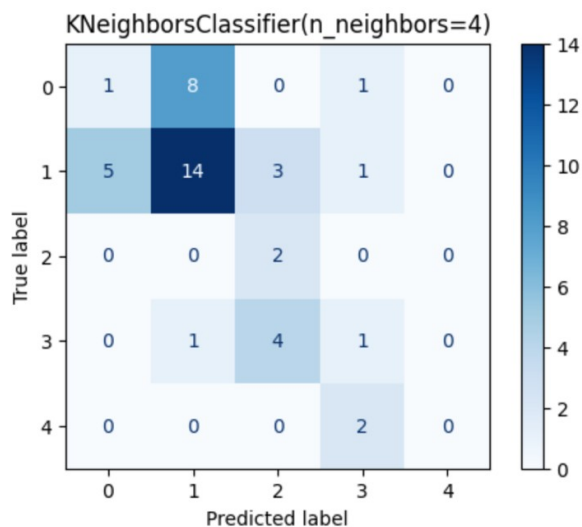
A consequence of the preprocessing is that the target variable has effectively become binary (0=no heart disease, 1=heart disease), as observations with higher severity values have been removed. This has simplified the classification task, as the models now only need to distinguish between the presence and absence of heart disease. In addition, the size of the dataset has decreased significantly, as the number of observations has fallen from 141 to 95, thereby increasing the risk of overfitting, particularly with more complex models such as the Random Forest. This highlights the importance of additional validation techniques.

Confusion matrices were created for each model to evaluate model performance. These matrices provide insight into correct and incorrect predictions per class, with the diagonal showing the correct predictions and the remaining values showing the misclassifications. The confusion matrix produced by the Dummy Classifier shows that all observations are classified as class 0. It can therefore be concluded that the model does not learn any patterns from the dataset and makes no distinction between the different classes.

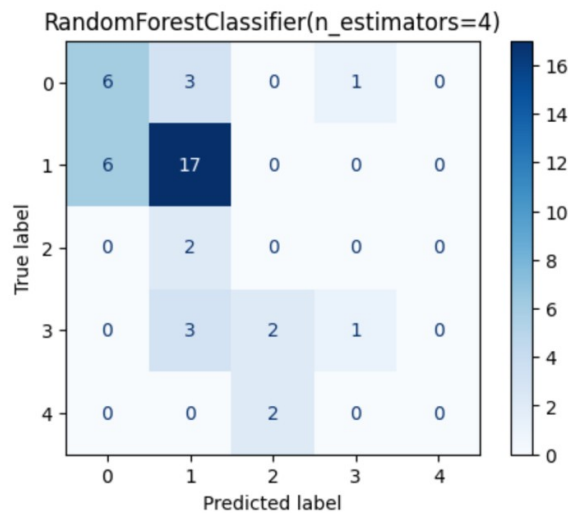
DummyClassifier(constant=0, strategy='constant')



The KNN model produces a confusion matrix that can predict multiple classes, but there are still a significant number of errors, which often occur between classes that are closely related. Observations from class 1 are regularly confused with those from class 0 or 2, which shows overlap in the feature space and are more difficult distinguished from one another based on the features.



In contrast, the Random Forest classifier shows a better distribution of correct predictions, with more values lying on the diagonal. This demonstrates that this model is better able to recognise underlying patterns in the dataset. Furthermore, the model continues to struggle with less common classes, as this is likely due to a poor distribution within the dataset.

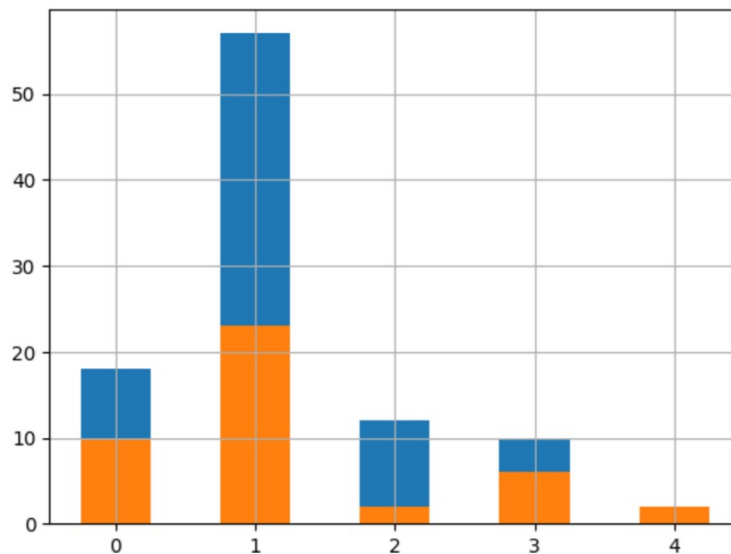


In addition to confusion matrices, other evaluation metrics are used, namely: accuracy, precision, recall and F1 score. However, whilst accuracy provides a general indication of the percentage of correct predictions, this metric is not particularly suitable for datasets with an uneven distribution. In this situation, recall is an important metric, as it indicates the extent to which the model correctly identifies patients with heart disease at all. False negatives can have serious consequences in a medical context, which is why the focus is on maximising recall. The F1-score, on the other hand, is used as a supplement to strike a balance between correctly identifying patients with the condition and minimising false predictions. The results show variation in the performance of classes. The lower recall for the less common class is less reliable due to the lack of observations. Furthermore, the dominant class performs better, demonstrating that bias is a consequence of “class imbalance”. This confirms that the use of multiple evaluation metrics is important for obtaining a complete picture of model performance.

	precision	recall	f1-score	support
0	0.17	0.10	0.12	10
1	0.61	0.61	0.61	23
2	0.22	1.00	0.36	2
3	0.20	0.17	0.18	6
4	0.00	0.00	0.00	2
accuracy			0.42	43
macro avg	0.24	0.38	0.26	43
weighted avg	0.40	0.42	0.40	43

The training and evaluation process has been standardised to enable reproducible and fair comparisons. All models were therefore trained under identical conditions and evaluated in the same way. The dataset has been split into 70% for training and 30% for testing, allowing the models to be tested on new data and providing a more realistic picture of how the models perform. The train-test split, as shown by the histogram, indicates that the class imbalance can be maintained in both the training and test sets, with one dominant class. This may therefore explain the superior performance of this class compared to the less common classes.

Save Histogram of Train/Test Label Counts ####
Histogram Saved



At the same time, 3-fold cross-validation was used, whereby the dataset was split into three parts and the model was trained and tested several times on different combinations of these splits. The final performance is determined by the average, resulting in a more robust and reliable evaluation. This approach results in a reproducible and partially automated experiment. It allows the models to be compared consistently, thereby providing a sound basis for further optimisation, such as testing different hyperparameters.

Discussion

The relationship between feature importance and data analysis

The feature importance derived from the Random Forest classifier largely corresponds to what we already observed in the data analysis from Task 1. This can be explained by the statistical properties of the features, such as variance, but also by the way a Random Forest model makes decisions.

The data analysis showed that certain numerical features, such as 'thalach', 'age', 'oldpeak' and 'trestbps', have a fairly wide range and high standard deviation. For example, 'thalach' has a standard deviation of 22.85 and a range from 60 to 184. This indicates that this feature varies greatly between patients, meaning it captures a lot of information to distinguish between the different levels of heart disease. The theory tells us that feature selection methods, such as VarianceThreshold in sklearn, often make features with low variance less useful, as they show little difference between the data points.

The Random Forest classifier determines feature importance based on how much a feature contributes to the splitting of data in the decision trees, by looking at the reduction in impurity, such as Gini impurity. Features that result in clear and consistent splits are given a higher importance score, as they reduce uncertainty in the predictions the most; consequently, it makes sense that features with a clear relationship and high variability with the target variable become more essential.

We see this reflected in features such as 'oldpeak' and 'thalach'. Due to the wide variation in "thalach", the model can draw boundaries between the different patient groups. In addition, "oldpeak" also has a clear medical significance, ST depression, making it even more relevant. This combination of substantive meaning and statistical variation ensures that these features contribute effectively to the model.

On the other hand, features such as "fbs", "restecg" and "sex" have low variance and often consist of values with limited variation or binary values. We saw in Task 1 that these features do not have much variation and are therefore less discriminatory. The theory also stated that features with lower variance are less informative. Given the nature of Random Forest, it therefore makes sense that these features are not used as frequently for splits and thus have a lower importance.

It is important to note, however, that feature importance does not depend solely on variance, as Random Forest can also detect complex and non-linear relationships between features. This indicates that a feature with lower variance can still sometimes be important if, in combination with other features, it provides meaningful information. In this case, however, the data analysis and the theoretical expectation point in the same direction, demonstrating that the model and the data are consistent with one another.

To summarise briefly: the feature importance from the Random Forest model aligns well with the statistical analysis from Task 1. Features that contain more variation and are closely related to heart disease in terms of content contribute most to the model's performance, whereas features with less variation have less influence. This demonstrates that the data analysis provides a solid basis for understanding how the model makes its decisions.

Enhancing the model's performance

One method of enhancing model performance involves addressing the dataset's imbalance. As was seen in the data exploration, the distribution of the target variable is skewed. Most of the patients belonged to one of the two groups (particularly, group one) with a few patients belonging to groups two and/or three. This degree of imbalance can lead to problems for the model because it will learn most of its patterns from the majority class and therefore, the model may not perform well when classifying instances from the minority classes. As such, the ability of the model to properly classify all groups may appear satisfactory even when the model is inaccurate at classifying those cases under-represented by their group.

To resolve this problem, we will implement resampling techniques, specifically oversampling, as the most appropriate approach for balancing the class distribution, primarily due to the relatively small size of our overall dataset. Oversampling increases the number of samples in the under-represented classes in order to create a more balanced distribution across the classes. This can be accomplished by creating duplicate copies of current samples or by creating new "synthetic" samples.

SMOTE is a common strategy used when attempting to oversample minority classes in an imbalanced dataset. Rather than creating copies of existing samples, SMOTE creates new synthetic samples by calculating the interpolation of a number of similar samples from the minor class. By training the model on these new samples, the model will learn a more generalised and meaningful representation of the behaviour of the minority class, rather than simply memorising duplicated data. As a result, the model will be able to more effectively distinguish between class labels.

Another method to address data imbalance is to remove samples from the majority class (undersampling). However, given that our dataset is relatively small, undersampling would result in the loss of a substantial amount of valuable data. In addition, removing a large number of samples from the majority class would negatively impact the model's validity and ability to generalise to new data. Therefore, oversampling will represent a better method of overcoming data imbalance in this case.

As a result of balancing the dataset, the model becomes less biased in favour of the majority class and is better able to identify patterns across all classes. This results in higher model performance, particularly when evaluating the minority classes based on recall and F1-score. Because accuracy alone is not an accurate representation of the model's performance when working with imbalanced datasets, recall and F1-score are the most appropriate measures of model performance when working with imbalanced datasets.

Ultimately, using oversampling techniques like SMOTE increases model reliability and fairness because it ensures that all classes are adequately represented in training data, helping the model to produce more accurate and fair predictions across heart disease severity levels.

Reflection on feedback received and improvements

Based on the feedback we received for Assignment 3, the 'Model Training' section has been improved to include a clearer explanation of the models used and an example of how decisions are made in a Random Forest. The choice of evaluation metrics has also been better justified, with the emphasis on recall and the F1 score. Finally, the training process has been described more clearly as a structured approach.